

YOLO Environment Setup

YOLO Environment Setup

JetsonNano

One-Click Installation Using Commands

Recommended Docker Startup Script (Universal for USB/CSI Cameras)

JetsonOrin Series

This chapter applies to manual setup on official image. If you are using the YAHBOOM version image, you can ignore this tutorial.

JetsonNano

Due to system version reasons, JetsonNano series needs to use the Docker image officially provided by ultralytics. The Yahboom version factory image already has the docker environment pre-installed, so no manual download is required. This section is only for those who want to manually download and install. Before starting, you need to configure the docker proxy environment to avoid network blocking

Note: It cannot be converted to engine format on JetsonNano because the current JetPack 4.6.3 / TensorRT 8.2.0.6 cannot directly parse Mod operators

One-Click Installation Using Commands

```
t=ultralytics/ultralytics:latest-jetson-jetpack4
sudo docker pull $t && sudo docker run -it --ipc=host --runtime=nvidia --gpus all
ultralytics/ultralytics:latest-jetson-jetpack4
```

```
root@32e0e9e11150:/ultralytics# ls
CITATION.cff  LICENSE  README.zh-CN.md  docs  mkdocs.yml  tests  ultralytics.egg-info
CONTRIBUTING.md  README.md  docker  examples  pyproject.toml  ultralytics  yolo26n.pt
root@32e0e9e11150:/ultralytics#
```

Use CLI prediction method to test

(Execute in the root@jetson-yahboom:/ultralytics directory)

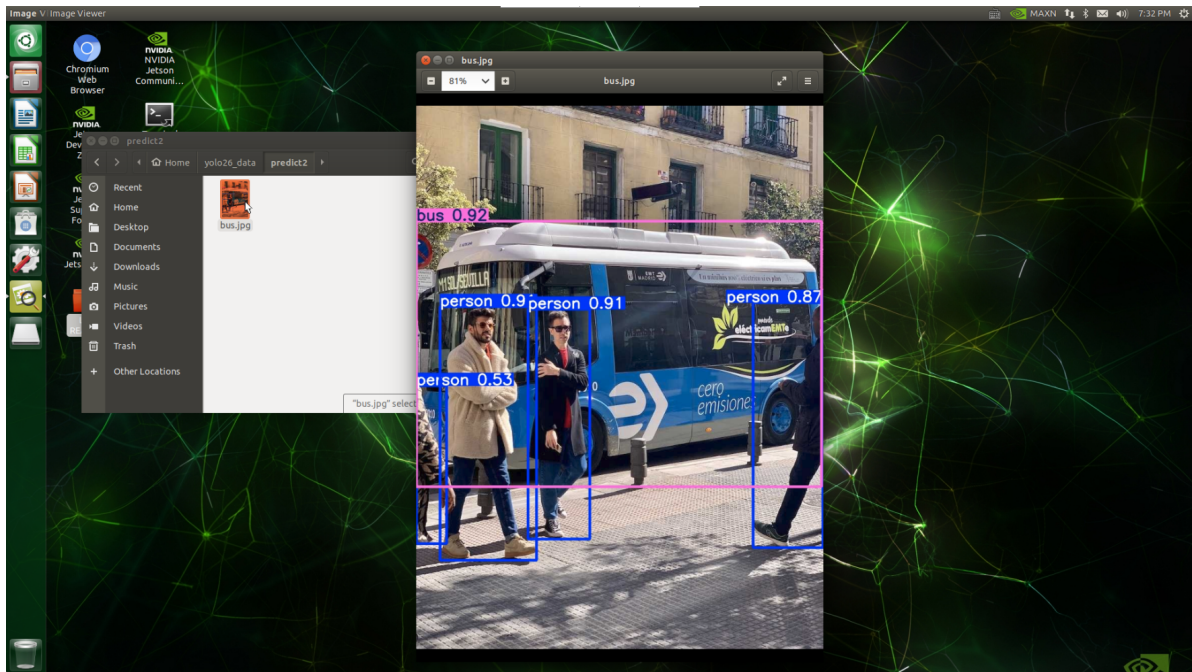
```
# Run inference with the exported model
yolo predict model=yolo26n.pt source='https://ultralytics.com/images/bus.jpg'

# If external network cannot be accessed, you can test with local images
yolo predict model=yolo26n.pt source='/root/yolo26_data/bus.jpg'
```

```
root@jetson-yahboom:/ultralytics# yolo predict model=yolo26n.pt source='/root/yolo26_data/bus.jpg'
Ultralytics 8.4.24 Python-3.8.0 torch-1.11.0a0+gitbc2c6ed CUDA:0 (NVIDIA Tegra X1, 3964MiB)
/ultralytics/ultralytics/nn/modules/head.py:246: UserWarning: _floordiv_ is deprecated, and its behavior will change in a future version of pytorch.
It currently rounds toward 0 (like the 'trunc' function NOT 'floor'). This results in incorrect rounding for negative values. To keep the current behavior, use torch.div(a, b, rounding_mode='trunc'), or for actual floor division, use torch.div(a, b, rounding_mode='floor').
idx = ori_index[torch.arange(batch_size)[..., None], index // nc] # original index
YOLO26n summary (fused): 122 layers, 2,408,932 parameters, 0 gradients, 5.4 GFLOPs

image 1/1 /root/yolo26_data/bus.jpg: 640x480 4 persons, 1 bus, 145.0ms
Speed: 16.9ms preprocess, 145.0ms inference, 15.5ms postprocess per image at shape (1, 3, 640, 480)
Results saved to /ultralytics/runs/detect/predict3
? Learn more at https://docs.ultralytics.com/modes/predict
```

We can copy the generated images to the ~/yolo26_data/ directory using the cp command, so they can be viewed on the host machine



Recommended Docker Startup Script (Universal for USB/CSI Cameras)

If you need to use USB cameras or CSI cameras in docker, it is recommended to use the following startup script:

```
#!/bin/bash
set -e

export DISPLAY=:0

xhost +local:root >/dev/null 2>&1 || true
xhost +local: >/dev/null 2>&1 || true

docker run -it \
  --net=host \
  --ipc=host \
  --privileged \
  --runtime=nvidia \
  --gpus all \
  -e DISPLAY=:0 \
  -e QT_X11_NO_MITSHM=1 \
  -e XAUTHORITY=/root/.Xauthority \
  -v ~/yolo26_data:/root/yolo26_data \
  -v ~/yahboom_demo:/ultraalytics/yahboom_demo \
  -v /tmp:/tmp \
  -v $HOME/.Xauthority:/root/.Xauthority:ro \
  -v /etc/nv_tegra_release:/etc/nv_tegra_release:ro \
  --device=/dev/video0 \
  --device=/dev/video1 \
  ultraalytics/ultraalytics:jetson-yahboom-custom \
  /bin/bash
```

Note: CSI cameras depend on `nvargus-daemon` in the Jetson system, so you need to mount `/tmp` so that the container can access `/tmp/argus_socket`

Note: USB cameras and CSI cameras can share the same docker startup script

JetsonOrin Series

If you are using Yahboom official image, please skip this step. yolo26 is already pre-installed in the factory image

1. Update package list, install pip and upgrade to latest version

```
sudo apt update
sudo apt install python3-pip -y
pip install -U pip
```

2. Install (update) `ultralytics` pip package with optional dependencies

```
pip install ultralytics[export]
```

3. Restart the device

```
sudo reboot
```

4. Download yolo26n.pt model

[Ultralytics YOLO26 - Ultralytics YOLO Documentation](#)

Performance Metrics							
Performance							
Detection (COCO) Segmentation (COCO) Classification (ImageNet) Pose (COCO) OBB (DO >							
See Detection Docs for usage examples with these models trained on COCO , which include 80 pretrained classes.							
Model	size (pixels)	mAP ^{val} 50-95	mAP ^{val} 50-95(e2e)	Speed CPU ONNX (ms)	Speed T4 TensorRT10 (ms)	params (M)	FLOPs (B)
YOLO26n	640	40.9	40.1	38.9 ± 0.7	1.7 ± 0.0	2.4	5.4
YOLO26s	640	48.6	47.8	87.2 ± 0.9	2.5 ± 0.0	9.5	20.7
YOLO26m	640	53.1	52.5	220.0 ± 1.4	4.7 ± 0.1	20.4	68.2
YOLO26l	640	55.0	54.4	286.2 ± 2.0	6.2 ± 0.2	24.8	86.4
YOLO26x	640	57.5	56.9	525.8 ± 4.0	11.8 ± 0.2	55.7	193.9

5. Convert to TensorRT format (accelerate inference)

After downloading, place it in the `/home/jetson/ultralytics/` path

Execute in terminal: `yolo export model=yolo26n.pt format=engine # creates 'yolo26n.engine'`

```

WARNING: A TensorRT requires GPU export, automatically assigning device=0
ultralytics 8.4.24 Python-3.10.12 torch-2.5.0asus728972e41.nv24.08 CUDA:0 (Orin, 15056MiB)
YOLO26n summary (fused): 122 layers, 2,408,932 parameters, 0 gradients, 5.4 GFLOPs

PyTorch: starting from 'yolo26n.pt' with input shape (1, 3, 640, 640) BCHW and output shape(s) (1, 300, 6) (5.3 M
B)

ONNX: starting export with onnx 1.17.0 opset 17...
/usr/local/lib/python3.10/dist-packages/torch/onnx/symbolic_opset9.py:5645: UserWarning: Exporting aten::index op
erator of advanced indexing in opset 17 is achieved by combination of multiple ONNX operators, including Reshape,
Transpose, Concat, and Gather. If indices include negative values, the exported graph will produce incorrect res
ults.
  warnings.warn(
ONNX: slimming with onnxslim 0.1.89...
ONNX: export success 2.3s, saved as 'yolo26n.onnx' (9.5 MB)

TensorRT: starting export with TensorRT 10.7.0...
[03/21/2026-12:24:54] [TRT] [I] [MemUsageChange] Init CUDA: CPU -2, GPU +0, now: CPU 818, GPU 5823 (MiB)
[03/21/2026-12:24:56] [TRT] [I] [MemUsageChange] Init builder kernel library: CPU +980, GPU +809, now: CPU 1838,
GPU 6667 (MiB)
[03/21/2026-12:24:56] [TRT] [I] -----
[03/21/2026-12:24:56] [TRT] [I] Input filename:      yolo26n.onnx
[03/21/2026-12:24:56] [TRT] [I] ONNX IR version:      0.0.8
[03/21/2026-12:24:56] [TRT] [I] Opset version:        17
[03/21/2026-12:24:56] [TRT] [I] Producer name:         pytorch
[03/21/2026-12:24:56] [TRT] [I] Producer version:    2.5.0
[03/21/2026-12:24:56] [TRT] [I] Domain:
[03/21/2026-12:24:56] [TRT] [I] Model version:        0
[03/21/2026-12:24:56] [TRT] [I] Doc string:
[03/21/2026-12:24:56] [TRT] [I] -----
TensorRT: input "images" with shape(1, 3, 640, 640) DataType:FLOAT
TensorRT: output "output0" with shape(1, 300, 6) DataType:FLOAT
TensorRT: building FP32 engine as yolo26n.engine
[03/21/2026-12:24:56] [TRT] [I] Local tuning cache in use. Profiling results in this builder pass will not be sto
red
[03/21/2026-12:26:29] [TRT] [I] Compiler backend is used during engine build.
[03/21/2026-12:27:44] [TRT] [I] [GraphReduction] The approximate region cut reduction algorithm is called.
[03/21/2026-12:27:44] [TRT] [I] Detected 1 inputs and 1 output network tensors.
[03/21/2026-12:27:46] [TRT] [I] Total Host Persistent Memory: 624656 bytes
[03/21/2026-12:27:46] [TRT] [I] Total Device Persistent Memory: 0 bytes
[03/21/2026-12:27:46] [TRT] [I] Max Scratch Memory: 2953728 bytes
[03/21/2026-12:27:46] [TRT] [I] [BlockAssignment] Started assigning block shifts. This will take 264 steps to complete.
[03/21/2026-12:27:46] [TRT] [I] [BlockAssignment] Algorithm ShiftNTopDown took 32.4306ms to assign 11 blocks to 264 nodes requiring 18253312 bytes.
[03/21/2026-12:27:46] [TRT] [I] Total Activation Memory: 18252800 bytes
[03/21/2026-12:27:46] [TRT] [I] Total Weights Memory: 9836992 bytes
[03/21/2026-12:27:46] [TRT] [I] Compiler backend is used during engine execution.
[03/21/2026-12:27:46] [TRT] [I] Engine generation completed in 170.499 seconds.
[03/21/2026-12:27:46] [TRT] [I] [MemUsageStats] Peak memory usage of TRT CPU/GPU memory allocators: CPU 1 MiB, GPU 50 MiB
TensorRT: export success 175.3s, saved as 'yolo26n.engine' (11.5 MB)

Export complete (176.7s)
Results saved to /home/jetson/ultralytics
Predict:      yolo predict task=detect model=yolo26n.engine imgsz=640
Validate:     yolo val task=detect model=yolo26n.engine imgsz=640 data=/home/lq/codes/ultralytics/ultralytics/cfg/datasets/coco.yaml
Visualize:    https://netron.app
v Learn more at https://docs.ultralytics.com/modes/export
jetson@phboom:~/ultralytics$

```